SUCH- und SORTIERALGO-RITHMEN

Coders.Eleven - JAVA

Coders.Eleven JAVA
Such- und
Sortieralgorithmen

#1 Lineare Suche

Annahmen

Idee

Implementierung

Analyse

Rekursive Variante

#2 Binäre Suche

Idee

Implementierung

Analyse

Typische Fehler

Rekursive Variante

Java API

#3 Vergleich

#4 Elementare Sortieralgorithmen

Annahmen

Bubblesort

Insertionsort

Selectionsort

Stabilität

#5 Effiziente Sortieralgorithmen

Divide and Conquer

Mergesort

Quicksort

Vergleich

#6 Sortieren in linearer Zeit

Annahmen

Countingsort



#1 Lineare Suche

Annahmen

Idee

Implementierung

Analyse

Rekursive Variante



Annahmen

- Allgemein
  - Daten werden in Arrays gespeichert
- Daten
  - Im Folgenden sind alle Beispiele mit ganzen Zahlen
  - Funktionieren auch auf anderen Datentypen
- Methoden
  - Es werden immer existierende Arrays übergeben (daher keine explizite Prüfung auf `null`)

Idee

- Suche nach einem Wert in einem unsortierten Array der Länge *n*
- Rückgabe
  - Position im Array (Index), an der der Wert gefunden wurde
  - Ungültiger Index, wenn nicht gefunden
    - Kann als Konstante festgelegt werden, zB. -1

Implementierung

```java
private int search(int[] data, int key) {
    for(int i = 0; i < data.length; i++) {
        if(data[i] == key) {
            return i;
        }
    }
    return NOT_FOUND;
}
```

Konstante, zB.
`private static final int NOT_FOUND = -1;`

Fallanalyse

- Bester Fall (Best Case)
  - Gesuchter Wert befindet sich an erster Position im Array
  - Nur ein Element muss untersucht werden
- Durchschnittlicher Fall (Average Case)
  - Bei Gleichverteilung, befindet sich der gesuchte Wert mit jeweils gleicher Wahrscheinlichkeit an jeder möglichen Stelle im Array
  - Durchschnittlich werden *n/2* Elemente untersucht
- Schlechtester Fall (Worst Case)
  - Gesuchter Wert befindet sich an letzter Position bzw. nicht im Array
  - Alle n Elemente müssen untersucht werden

Analyse

- Praxis: Wir gehen vom Worst Case aus
  - Oft wird nach nicht vorhandenen Werten gesucht
- Laufzeit von *O(n)*

Rekursive Variante

```java
private int searchRecursive(int[] data, int key) {
    return searchRecursive(data,  n: 0, key);
}

private int searchRecursive(int[] data, int n, int key) {
    if(n == data.length) {
        return NOT_FOUND;
    } else if(data[n] == key) {
        return n;
    } else {
        return searchRecursive(data,  n: n + 1, key);
    }
}
```

#2 Binäre Suche

Idee

Implementierung

Analyse

Typische Fehler

Rekursive Variante

Java API



Binäre Suche

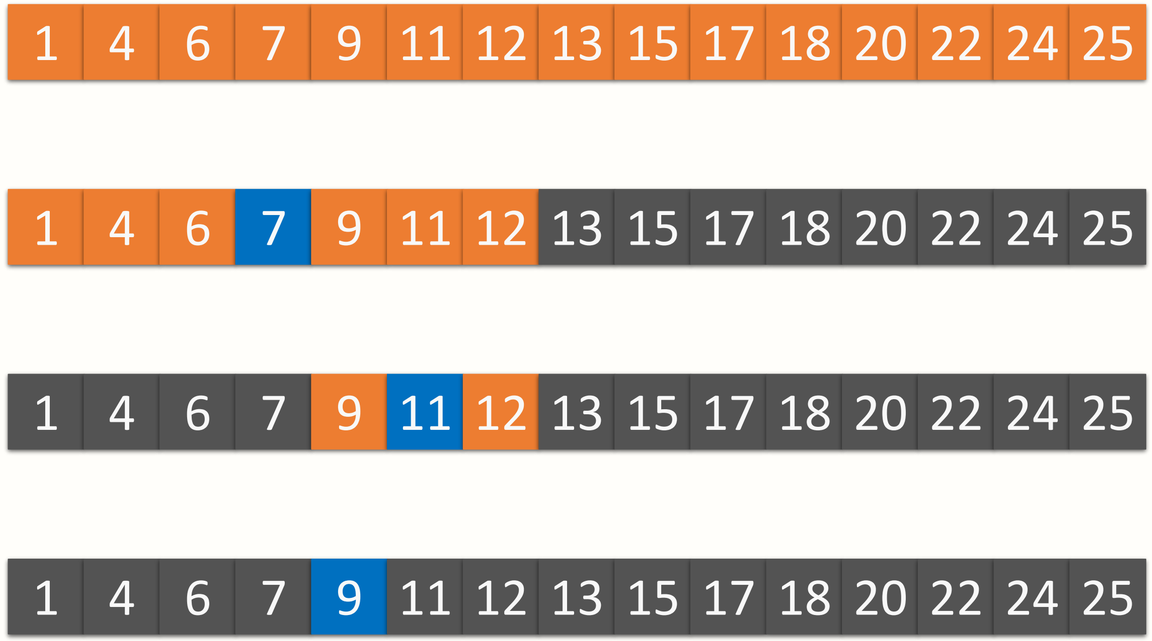
- Suche nach einem Wert in einem sortierten Array der Länge *n*
  - Lineare Suche möglich, jedoch ineffizient
  - Sortierung ausnutzen
- Rückgabe
  - Position im Array (Index), an der der Wert gefunden wurde
  - Ungültiger Index, wenn nicht gefunden
    - Kann als Konstante festgelegt werden, zB. -1

Idee

- Überprüfe das Element in der Mitte des sortierten Arrays
  - Ist es gleich dem gesuchten Element, ist die Suche beendet
  - Ist es kleiner als das gesuchte Element
    - Suche in der rechten Hälfte weiter und ignoriere die linke Hälfte
  - Ist es größer als das gesuchte Element
    - Suche in der linken Hälfte weiter und ignoriere die rechte Hälfte
- In der zu untersuchenden Hälfte (und deren Hälften usw.) wird genauso vorgegangen
  - Die Länge des Suchbereichs wird mit jedem Schritt halbiert
  - Die Suche endet, wenn das Element gefunden wurde bzw. der Suchbereich leer ist

Beispiel

- Befindet sich 9 im Array?



Implementierung

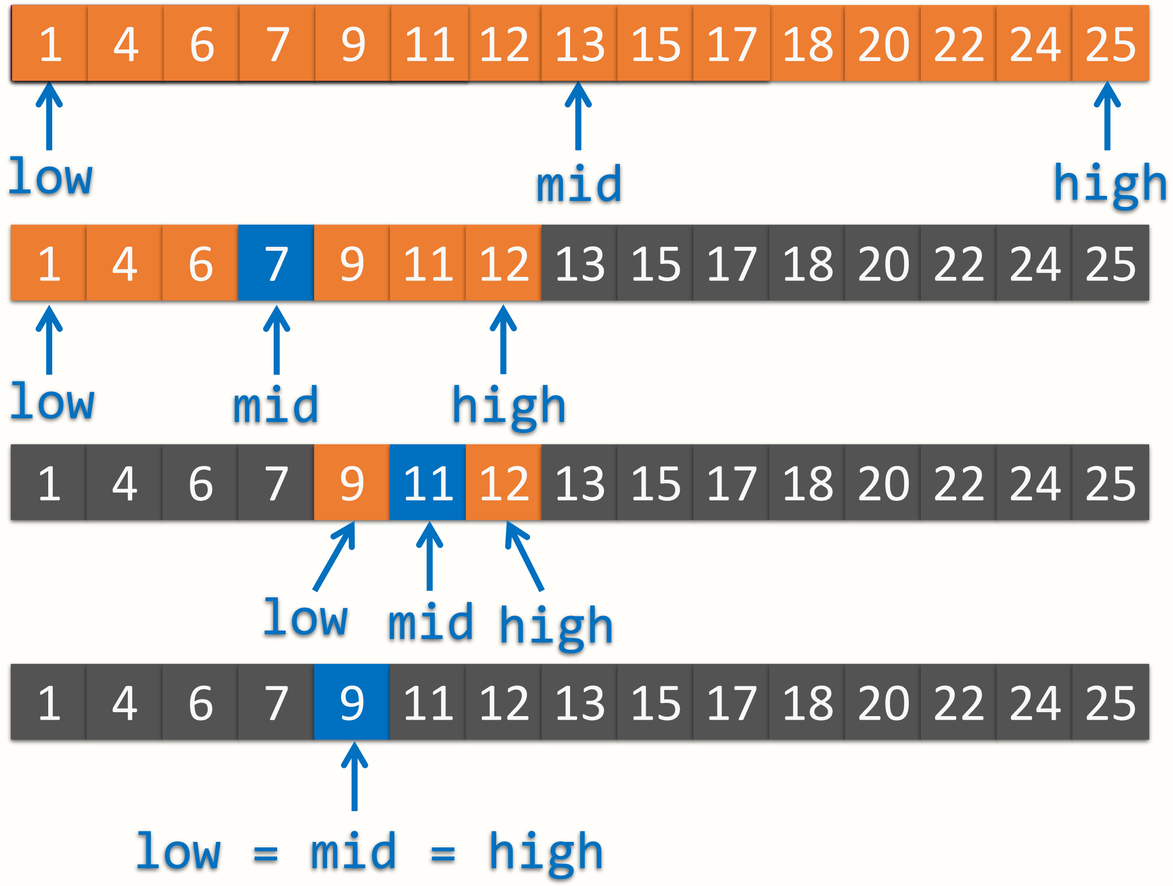
```java
private int binarySearch(int[] data, int key) {
    int low = 0, high = data.length - 1;
    while(low <= high) {
        int mid = low + (high - low) / 2;
        int value = data[mid];
        if(value < key) {
            low = mid + 1;
        } else if(value > key) {
            high = mid - 1;
        } else {
            return mid;
        }
    }
    return NOT_FOUND;
}
```

Vermeidet Überlauf

Konstante, zB. -1

Beispiel mit Variablen

- Befindet sich 9 im Array?



Analyse

- Bester Fall
  - Das Element wird im ersten Durchlauf (in der Mitte des Arrays) gefunden
- Durchschnittlicher Fall und schlechtester Fall
  - Bis zu einem Bereich der Größe 1 suchen
  - Laufzeit O(log n)

Schlechtester Fall	
n	**Anzahl der Durchläufe**
1	1
10	4
100	7
1.000	10
10.000	14
100.000	17
1.000.000	20
10.000.000	24
100.000.000	27
1.000.000.000	30

Typische Fehler

- Offensichtliche Variante für die Berechnung der Mitte
  - `int mid = (low + high)/2;`
  - Kann Überlauf produzieren
  - Bekannter Fehler (auch einige Jahre in der Java-Bibliothek)
- Grenzen des Suchbereichs
  - `mid` statt `mid + 1` bzw. `mid – 1`
  - Kann zu Endlosschleifen führen
- Schleifenbedingung
  - Statt $\texttt{low} \leq \texttt{high}$ nur $\texttt{low} < \texttt{high}$
  - Es werden dann nicht alle Einträge gefunden

Rekursive Variante

```java
private int binarySearchRecursive(int[] data, int key) {
    return binarySearchRecursive(data, key, low: 0, high: data.length - 1);
}
private int binarySearchRecursive(int[] data, int key, int low, int high) {
    if(low > high) {
        return NOT_FOUND;
    }
    int mid = low + (high - low) / 2;
    int value = data[mid];
    if(value < key) {
        return binarySearchRecursive(data, key, low: mid + 1, high);
    } else if(value > key) {
        return binarySearchRecursive(data, key, low, high: mid - 1);
    } else {
        return mid;
    }
}
```

Laufzeit im schlechtesten Fall wieder *O(log n)*

Binäre Suche auf Strings

```java
private int binarySearch(String[] data, String key) {
    int low = 0, high = data.length - 1;
    while(low <= high) {
        int mid = low + (high - low) / 2;
        int comp = data[mid].compareTo(key);
        if(comp < 0) {
            low = mid + 1;
        } else if(comp > 0) {
            high = mid - 1;
        } else {
            return mid;
        }
    }
    return NOT_FOUND;
}
```

Vergleich der Form
string1.compareTo(string2)
Rückgabe:

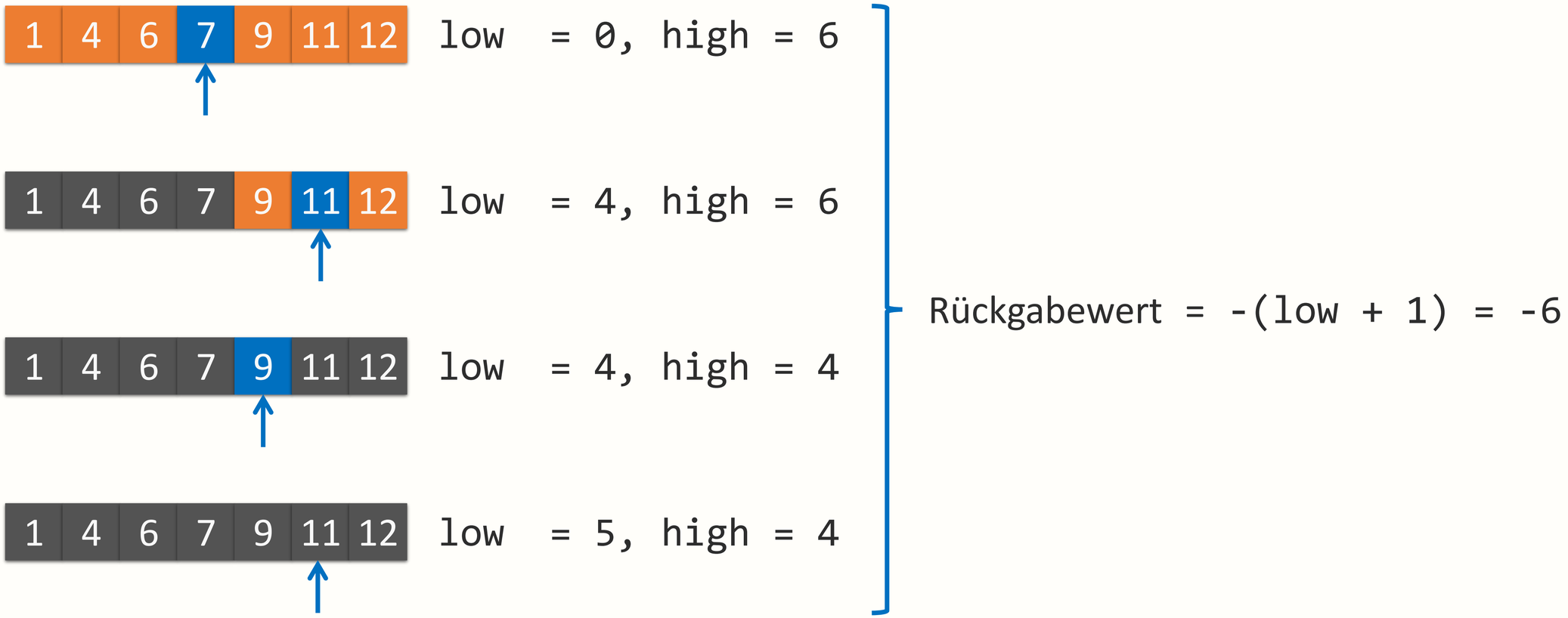
- *0 – beide Stings (Inhalt) sind gleich*
- *< 0 – string1 kleiner als string2*
- *> 0 – string1 größer als string2*

Rückgabewerte in der Java-API

- Berechnung des Rückgabewerts, wenn nicht gefunden mit `-(low + 1)`
- Idee
  - Negativer Wert zeigt an, dass der gesuchte Wert nicht gefunden wurde
  - Aus dem Rückgabewert kann die Stelle rekonstruiert werden, an der der Wert in der sortierten Reihe stehen würde

Beispiel

- Erfolglose Suche nach dem Wert 10



#3 Vergleich



Lineare Suche vs. Binäre Suche

- Sortierung
  - Lineare Suche funktioniert auf unsortierten und sortierten Arrays
  - Binäre Suche funktioniert nur auf sortierten Arrays
- Laufzeit
  - Lineare Suche in *O(n)*
  - Binäre Suche in *O(log n)*
- Gesuchter Wert kommt mehrfach im Array vor
  - Lineare Suche findet den Array-Eintrag mit dem kleinsten Index
  - Binäre Suche findet den Array-Eintrag (von mehreren) mit dem gesuchten Wert

Wann binäre Suche vorziehen?

- Wenn die Daten schon sortiert sind
- Wenn die Daten nur einmal sortiert werden müssen, aber die Suche sehr oft durchgeführt wird
  - Die Kosten der Sortierung (*SORT*) treten nur einmal auf
  - Die Kosten für das Suchen treten *k*-mal auf
  - Binäre Suche verwenden, wenn
    - $SORT + k * O(\log n) < k * O(n)$

      binär linear
    - Für große *k* wird der Aufwand für die Sortierung (*SORT*) irrelevant

#4 Elementare Sortieralgorithmen



Annahmen

Bubblesort

Insertionsort

Selectionsort

Stabilität

Annahmen

- Allgemein
  - Daten (sog. Schlüssel) werden in Arrays gespeichert
  - Auf den Daten ist eine Ordnungsrelation „<=“ definiert
- Daten
  - Im Folgenden sind alle Beispiele mit ganzen Zahlen
  - Funktionieren auch auf anderen Datentypen
- Methoden
  - Es werden immer existierende Arrays übergeben (daher keine explizite Prüfung auf `null`)
  - O.B.d.A. wird immer aufsteigend sortiert

Bubblesort – Idee

- Durchlauf
  - Array wird von links nach rechts durchlaufen
  - Es werden immer zwei benachbarte Elemente betrachtet
  - Stehen diese in falscher Reihenfolge, so werden sie vertauscht
- Wird so oft wiederholt, bis das Array komplett sortiert ist

Bubblesort – Implementierung

```
private void bubbleSort(int[] data) {
    for(int i = 0; i < data.length - 1; i++) {
        for(int j = 0; j < data.length - 1; j++) {
            if(data[j] > data[j + 1]) {
                exchange(data, j, j + 1);
            }
        }
    }
}
private void exchange(int[] a, int i, int j) {
    int swap = a[i];
    a[i] = a[j];
    a[j] = swap;
}
```

Bubblesort – Beispiel (1)

- Array: `[73, 2, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
- Darstellung der inneren Schleife für `i = 0`:
  - `[2, 73, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
  - `[2, 19, 73, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
  - `[2, 19, 73, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
  - `[2, 19, 73, 11, 96, 5, 73, 66, 99, 1, 72, 30]`
  - `[2, 19, 73, 11, 5, 96, 73, 66, 99, 1, 72, 30]`
  - `[2, 19, 73, 11, 5, 73, 96, 66, 99, 1, 72, 30]`
  - `[2, 19, 73, 11, 5, 73, 66, 96, 99, 1, 72, 30]`
  - `[2, 19, 73, 11, 5, 73, 66, 96, 99, 1, 72, 30]`
  - `[2, 19, 73, 11, 5, 73, 66, 96, 1, 99, 72, 30]`
  - `[2, 19, 73, 11, 5, 73, 66, 96, 1, 72, 99, 30]`
  - `[2, 19, 73, 11, 5, 73, 66, 96, 1, 72, 30, 99]`

```java
private void bubbleSort(int[] data) {
    for(int i = 0; i < data.length - 1; i++) {
        for(int j = 0; j < data.length - 1; j++) {
            if(data[j] > data[j + 1]) {
                exchange(data, j, j + 1);
            }
        }
    }
}
```

Nach einem kompletten Durchlauf steht das Maximum ganz rechts

Bubblesort – Beispiel (2)



Bubblesort – Analyse

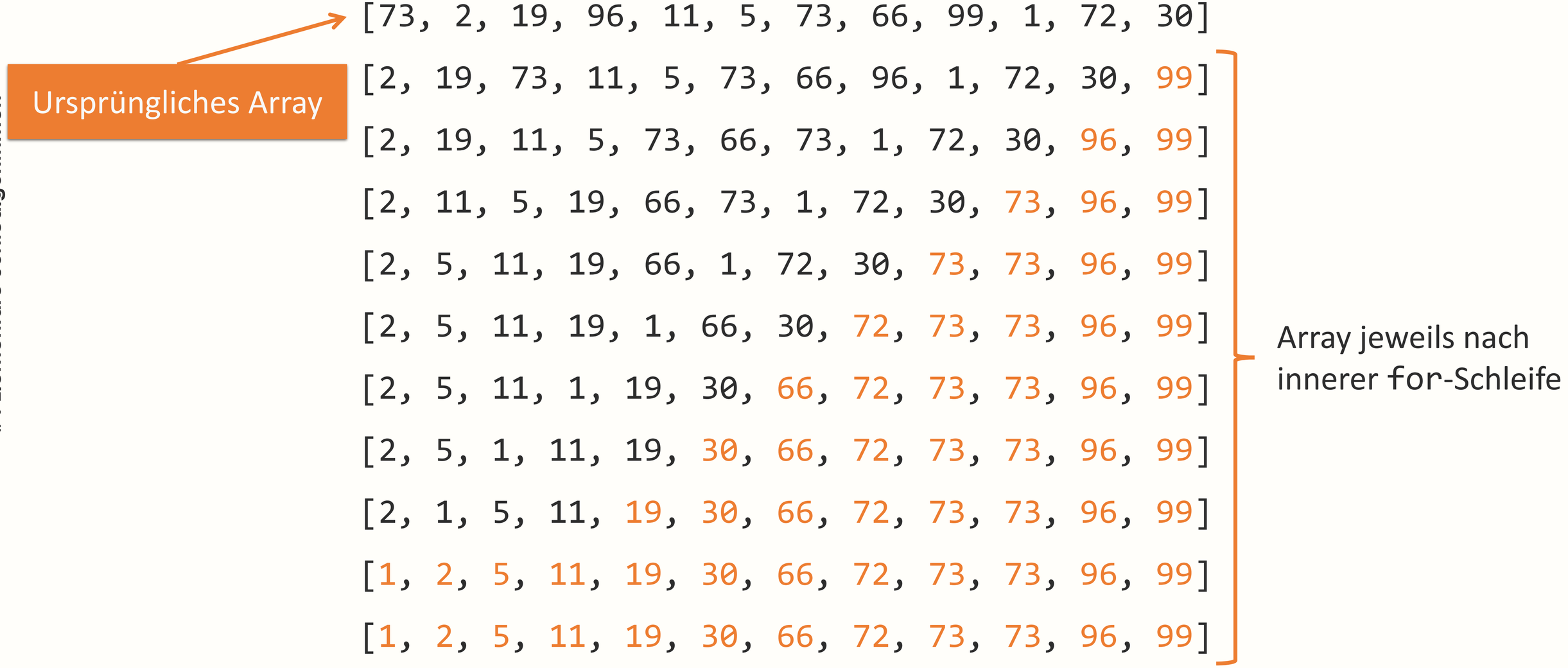
- Zwei verschachtelte Schleifen werden immer komplett durchlaufen
- Laufzeit (Aufwand) wird beschreiben mit „O“
  - Laufzeit von Bubblesort ist $O(n^2)$
  - „Quadratischer Aufwand“ – nicht effizient

Bubblesort – Optimierung

```
private void bubbleSort2(int[] data) {
    boolean notSorted = true;
    while(notSorted) {
        notSorted = false;
        for(int i = 0; i < data.length - 1; i++) {
            if(data[i] > data[i + 1]) {
                exchange(data, i,  j: i + 1);
                notSorted = true;
            }
        }
    }
}
```

Abbruch, wenn in einem Durchlauf keine Elemente mehr vertauscht werden

Bubblesort optimiert – Beispiel

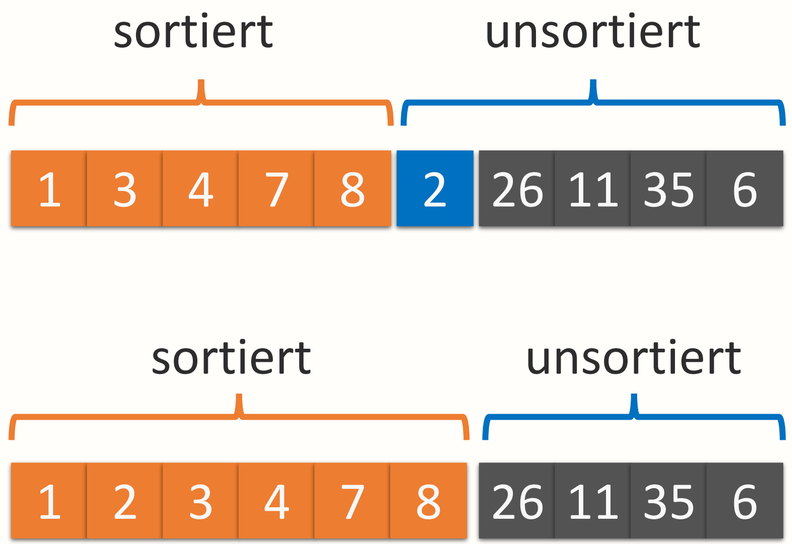


Bubblesort optimiert – Analyse

- Schlechtester Fall (Array ist absteigend sortiert)
  - Es müssen immer beide Schleifen durchlaufen werden
  - Laufzeit ist $O(n^2)$
- Durchschnittlicher Fall
  - Wie schlechtester Fall
- Bester Fall (Array ist aufsteigend sortiert)
  - Optimierte Variante durchläuft Array einmal
  - Laufzeit ist $O(n)$

Insertionsort – Idee

- Nimmt aus dem unsortierten Teil der Eingabesequenz das erste Element
- Fügt das Element an der richtigen Stelle in die bisher sortierte Sequenz ein



Insertionsort – Implementierung

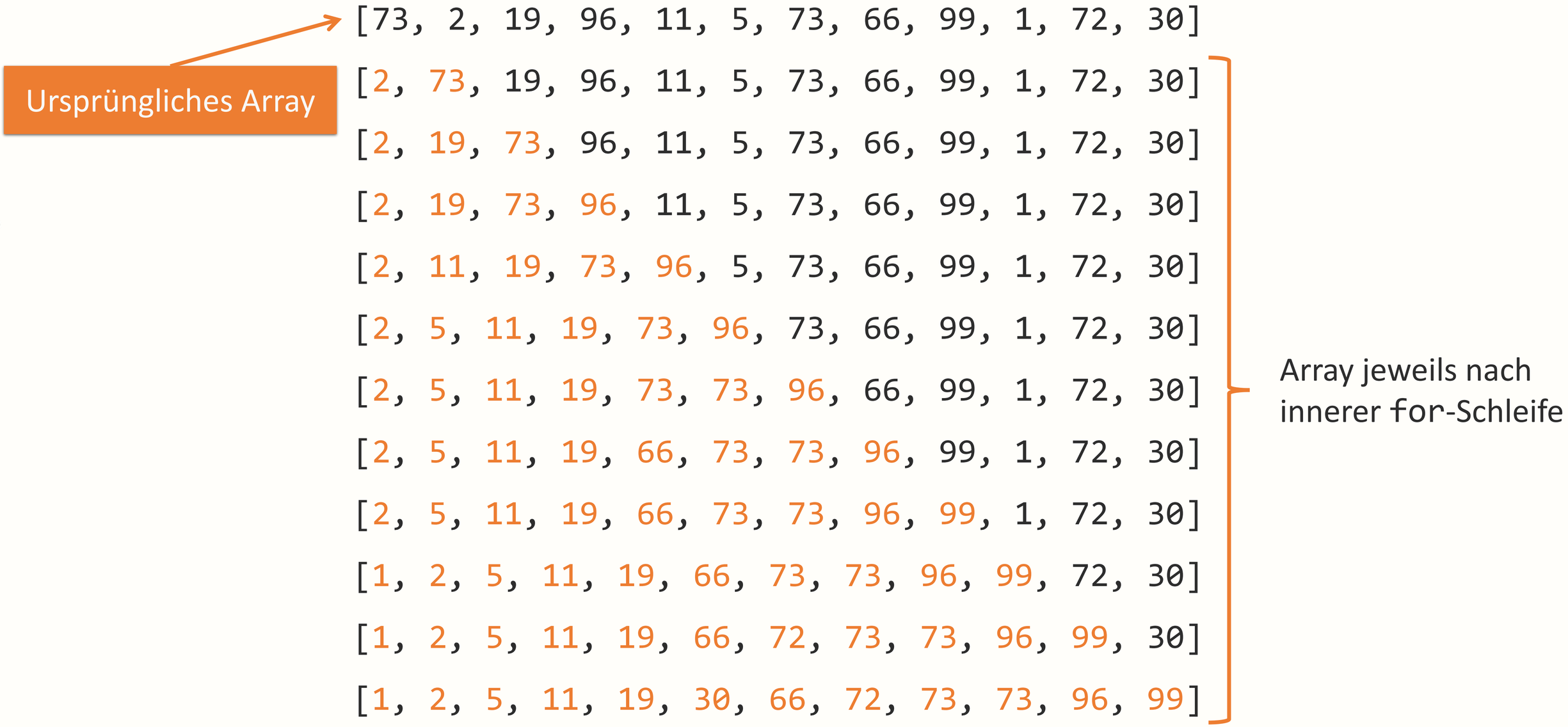
```
private void insertionSort(int[] data) {
    for(int i = 1; i < data.length; i++) {
        for(int j = i; j > 0 && data[j] < data[j - 1]; j--) {
            exchange(data, j, j - 1);
        }
    }
}
```

Insertionsort – Beispiel (1)

- Ursprüngliches Array: `[73, 2, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
- Innere Schleife für `i = 1`:
  - `[2, 73, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
- Innere Schleife für `i = 2`:
  - `[2, 19, 73, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
- Innere Schleife für `i = 4`:
  - `[2, 19, 73, 11, 96, 5, 73, 66, 99, 1, 72, 30]`
  - `[2, 19, 11, 73, 96, 5, 73, 66, 99, 1, 72, 30]`
  - `[2, 11, 19, 73, 96, 5, 73, 66, 99, 1, 72, 30]`

```
private void insertionSort(int[] data) {
    for(int i = 1; i < data.length; i++) {
        for(int j = i; j > 0 && data[j] < data[j - 1]; j--) {
            exchange(data, j, j - 1);
        }
    }
}
```

Insertionsort – Beispiel (2)

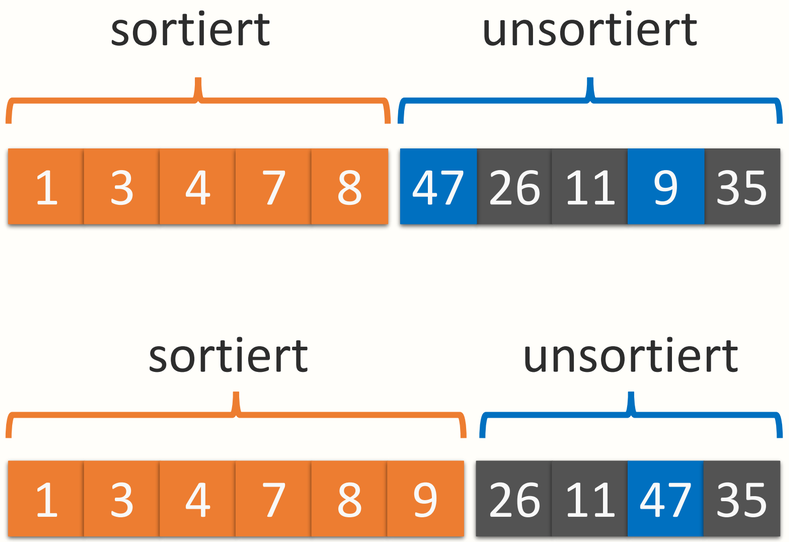


Insertionsort – Analyse

- Schlechtester Fall (Array ist absteigend sortiert)
  - Schleifen werden komplett durchlaufen
  - Laufzeit ist $O(n^2)$
- Durchschnittlicher Fall
  - Wie schlechtester Fall
- Bester Fall (Array ist aufsteigend sortiert)
  - Innere Schleife wird nie ausgeführt
  - Laufzeit ist $O(n)$

Selectionsort – Idee

- Nimmt das kleinste Element aus dem unsortierten Teil
- Tauscht das Element an der ersten Stelle nach dem sortierten Teil mit dem kleinsten Element



Selectionsort – Implementierung

```
private void selectionSort(int[] data) {
    for(int i = 0; i < data.length - 1; i++) {
        int min = i;
        for(int j = i + 1; j < data.length; j++) {
            if(data[j] < data[min]) {
                min = j;
            }
        }
        exchange(data, i, min);
    }
}
```

Selectionsort – Beispiel (1)

- Ursprüngliches Array: `[73, 2, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]`
- Innere Schleife für `i = 0`:
  - Index des Minimums = 9
  - `[1, 2, 19, 96, 11, 5, 73, 66, 99, 73, 72, 30]`
- Innere Schleife für `i = 1`:
  - Index des Minimums = 1
  - `[1, 2, 19, 96, 11, 5, 73, 66, 99, 73, 72, 30]`
- Innere Schleife für `i = 2`:
  - Index des Minimums = 5
  - `[1, 2, 5, 96, 11, 19, 73, 66, 99, 73, 72, 30]`

Selectionsort – Beispiel (2)

Ursprüngliches Array

```
[73, 2, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]
[1, 2, 19, 96, 11, 5, 73, 66, 99, 73, 72, 30]
[1, 2, 19, 96, 11, 5, 73, 66, 99, 73, 72, 30]
[1, 2, 5, 96, 11, 19, 73, 66, 99, 73, 72, 30]
[1, 2, 5, 11, 96, 19, 73, 66, 99, 73, 72, 30]
[1, 2, 5, 11, 19, 96, 73, 66, 99, 73, 72, 30]
[1, 2, 5, 11, 19, 30, 73, 66, 99, 73, 72, 96]
[1, 2, 5, 11, 19, 30, 66, 73, 99, 73, 72, 96]
[1, 2, 5, 11, 19, 30, 66, 72, 99, 73, 73, 96]
[1, 2, 5, 11, 19, 30, 66, 72, 73, 99, 73, 96]
[1, 2, 5, 11, 19, 30, 66, 72, 73, 73, 99, 96]
[1, 2, 5, 11, 19, 30, 66, 72, 73, 73, 96, 99]
```

Array jeweils nach innerer `for`-Schleife

Selectionsort – Analyse

- Zwei verschachtelte Schleifen werden immer komplett durchlaufen
  - Laufzeit ist $O(n^2)$
  - Bester Fall, durchschnittlicher und schlechtester Fall sind gleich
- Anzahl der Aufrufe von `exchange`
  - Linear, unabhängig von der Eingabesequenz
  - Insertionsort: Abhängig von der Eingabesequenz
    - Kann dort auch quadratisch sein

Vergleich

- Bubblesort
  - Einfach zu implementieren
- Insertionsort
  - Lineare Laufzeit bei vorsortierten Sequenzen
- Selectionsort
  - Anzahl der Vertauschungen steigt linear an
  - Anzahl der Vergleiche quadratisch

Stabilität

- Stabiles Sortierverfahren
  - Reihenfolge der Datensätze (komplexe Datentypen), deren Sortierschlüssel gleich sind, bleibt bewahrt
- Beispiel für komplexen Datentyp:
  - Personaldaten
    - Name
    - Abteilungsnummer
  - Beide Informationen werden gemeinsam gespeichert

Stabilität – Beispiel (1)

- Liste von Personaldaten mit Abteilungsnummer
  - Innerhalb der Abteilung sortiert nach Vornamen

3 Daniel	4 Maria	2 Paul	3 Erika	1 Anton	3 Sarah

- Sortierung nach Abteilungsnummer mit stabilem Sortierungsverfahren
  - Sortierung innerhalb einer Abteilung bleibt erhalten

1 Anton	2 Paul	3 Daniel	3 Erika	3 Sarah	4 Maria

Stabilität – Beispiel (2)

- Sequenz vereinfacht: 3 4 2 3 1 3
- Vergleich mit den bisherigen Algorithmen
  - Ausgaben nach inneren Schleifen

Bubblesort	Insertionsort	Selectionsort
3 4 2 3 1 3	3 4 2 3 1 3	3 4 2 3 1 3
3 2 3 1 3 4	3 4 2 3 1 3	1 4 2 3 3 3
2 3 1 3 3 4	2 3 4 3 1 3	1 2 4 3 3 3
2 1 3 3 3 4	2 3 3 4 1 3	1 2 3 4 3 3
1 2 3 3 3 4	1 2 3 3 4 3	1 2 3 3 4 3
1 2 3 3 3 4	1 2 3 3 3 4	1 2 3 3 3 4
stabil	stabil	nicht stabil

Vergleich – Laufzeiten

- Arrays der Länge *n* mit Zufallszahlen im Bereich *0, …,n*
- Laufzeiten für einen Algorithmus
  - Mittelwert von zehn Messungen pro Eingabegröße
- System: Intel Core i7 9750H, Windows 10, Java 14

Eingabegröße n	Bubblesort (sec)	Insertionsort (sec)	Selectionsort (sec)
10.000	0,10	0,01	0,03
20.000	0,43	0,05	0,09
40.000	1,80	0,19	0,36
80.000	7,28	0,79	1,42
160.000	29,17	3,14	5,66

#5 Effiziente Sortieralgorithmen

Divide and Conquer

Mergesort

Quicksort

Vergleich



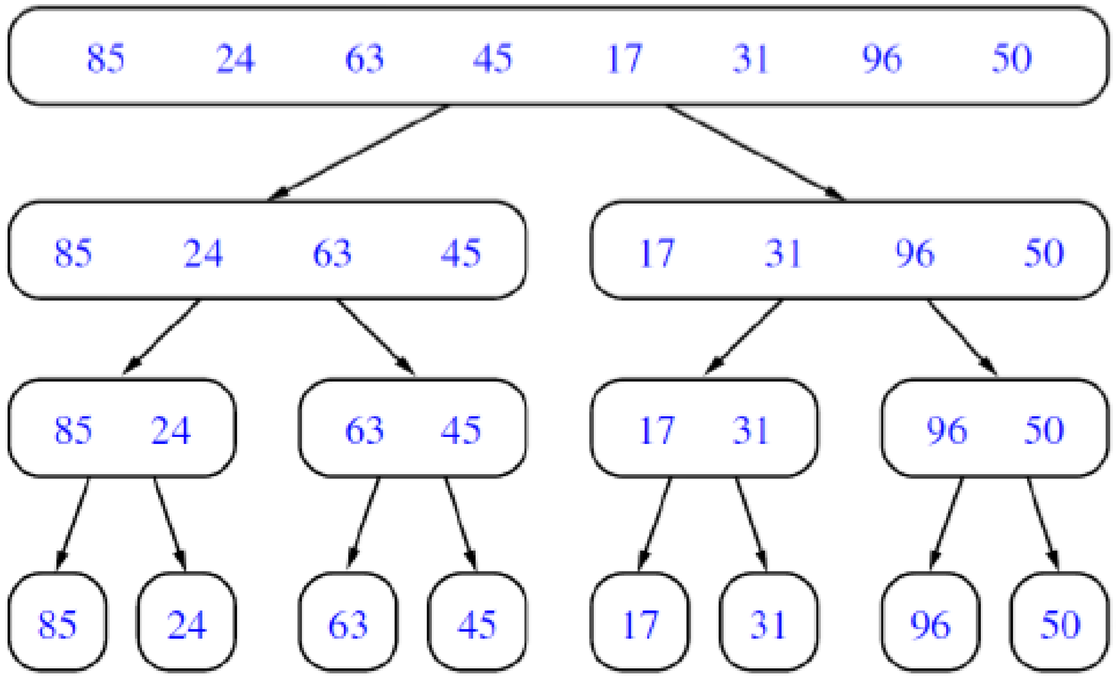
Divide and Conquer

- Deutsch: Teile-und-herrsche-Verfahren
- Allgemeines Prinzip für Algorithmenentwurf
  - Teile ein Problem in Teilprobleme auf
  - Löse jedes Teilproblem unabhängig für sich
  - Teillösungen zu einer Gesamtlösung für das ursprüngliche Problem kombinieren

Mergesort – Idee

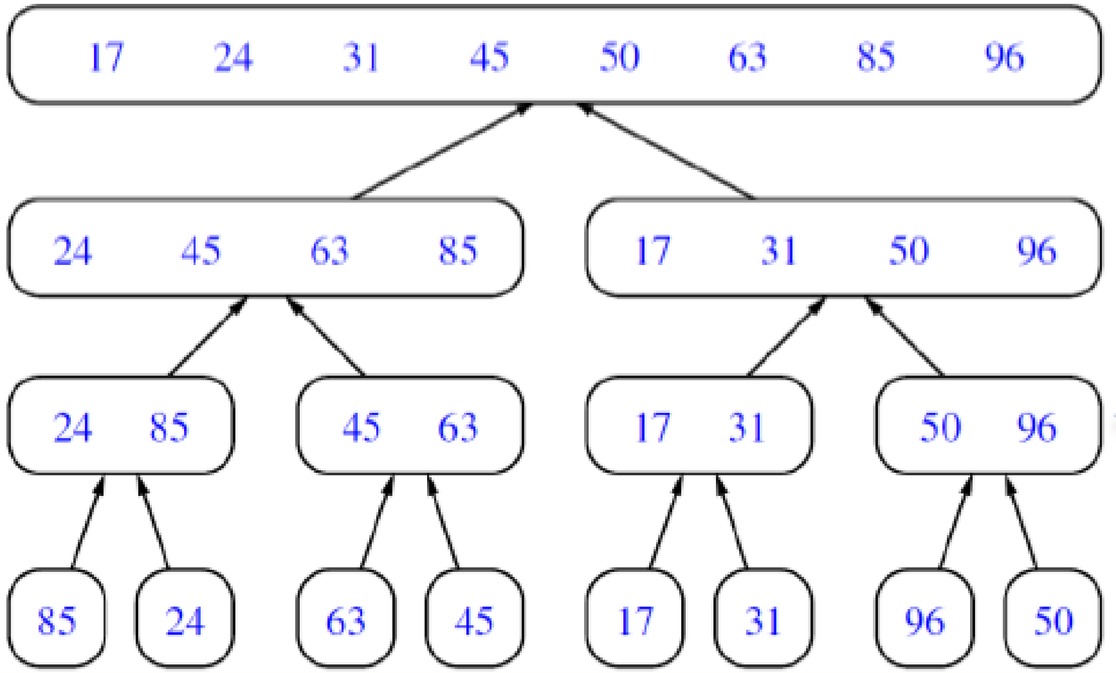
- Divide and Conquer
- Vorgehen:
  - Ursprüngliche Menge an Daten wird in zwei Hälften geteilt
  - Sortiere jede Hälfte für sich (rekursiv)
  - Verschmelze die beiden sortierten Hälften
    - Vergleiche immer wieder die aktuell vordersten Elemente der beiden sortierten Hälften
    - Entnimm das kleinere der beiden
    - Am Ende ergibt sich eine geordnete Sequenz aus den zwei geordneten Hälften
  - Wird auch als Top-Down-Mergesort bezeichnet

Mergesort – Ablauf



Solange aufteilen, bis nur mehr ein Element betrachtet wird

Verschmelzen von jeweils zwei sortierten Bereichen zu einem neu sortierten Bereich



Mergesort – Implementierung


```java
private void mergeSort(int[] data) {
    int[] help = new int[data.length];
    mergeSort(data, help, lo: 0, hi: data.length - 1);
}
private void mergeSort(int[] data, int[] help, int lo, int hi) {
    if(hi <= lo) return;
    int mid = lo + (hi - lo) / 2;
    mergeSort(data, help, lo, mid);
    mergeSort(data, help, lo: mid + 1, hi);
    merge(data, help, lo, mid, hi);
}
private void merge(int[] data, int[] help, int lo, int mid, int hi) {
    for(int k = lo; k <= hi; k++) {
        help[k] = data[k];
    }
    int i = lo, j = mid + 1;
    for(int k = lo; k <= hi; k++) {
        if(i > mid) data[k] = help[j++];
        else if(j > hi) data[k] = help[i++];
        else if(help[j] < help[i]) data[k] = help[j++];
        else data[k] = help[i++];
    }
}
```

Aufruf mit einem Parameter

Mergesort:
Zuerst linke Hälfte sortieren, dann rechte Hälfte sortieren, zuletzt sortierte Hälften verschmelzen

Merge
Verschmelzen zweier sortierter Bereiche

Warning, da dies auch mit System.arraycopy() optimiert werden kann

Mergesort – Analyse

- Laufzeit ist immer *O(n * log(n))*
  - Mehr Speicherplatzbedarf durch Hilfsarray beim Verschmelzen
  - *n * log(n)* kann für das allgemeine Sortierproblem nicht unterboten werden
    - Gilt für vergleichsbasierte Verfahren, d.h. keine zusätzlichen Informationen über den Wertebereich bekannt
      - Sortieren in linearer Zeit: Counting Sort, Radix Sort
    - Es müssen *n* Werte sortiert werden
- Vorsortiertes Array bringt keinen Vorteil
  - Array wird immer geteilt und verschmolzen
- Stabiles Sortierverfahren

Mergesort – Alternative

- Es werden ausschließlich merge-Operationen durchgeführt
  - Zuerst 1er-Blöcke verschmelzen, dann 2er-Blöcke verschmelzen, dann 4er-Blöcke, usw.
- Bezeichnet man als Bottom-Up-Mergesort

```
private void mergeSortBU(int[] data) {
    int[] help = new int[data.length];
    for(int len = 1; len < data.length; len *= 2) {
        for(int lo = 0; lo < data.length - len; lo += len + len) {
            int mid = lo + len - 1;
            int hi = Math.min(lo + len + len - 1, data.length - 1);
            merge(data, help, lo, mid, hi);
        }
    }
}
```

Quicksort – Idee

- Divide and Conquer
- Vorgehen:
  - Wähle ein Element *x* (Pivotelement) aus dem Array
  - Teile den Abschnitt im Array in zwei Abschnitte A_1 und A_2, so dass gilt:
    - A_1 enthält nur Elemente kleiner gleich *x*
    - A_2 enthält nur Elemente größer gleich *x*
  - Rekursive Aufrufe
    - Rekursiver Aufruf A_1 erzeugt sortierte Folge B_1
    - Rekursiver Aufruf A_2 erzeugt sortierte Folge B_2
    - Ergebnis am Ende im Array auf einer Rekursionsebene
      - B_1, *x*, B_2

Quicksort – Implementierung

```
private void quickSort(int[] data) {
    quickSort(data, lo: 0, hi: data.length - 1);
}
private void quickSort(int[] data, int lo, int hi) {
    if(hi <= lo) return;
    int j = partition(data, lo, hi);
    quickSort(data, lo, hi: j - 1);
    quickSort(data, lo: j + 1, hi);
}
private int partition(int[] data, int lo, int hi) {
    int k = lo, v = data[hi];
    for(int i = k; i < hi; i++) {
        if(data[i] < v) {
            exchange(data, i, k++);
        }
    }
    exchange(data, k, hi);
    return k;
}
```

Aufruf mit einem Parameter

Quicksort:
Zuerst partitionieren, dann linken Teilbereich sortieren, dann rechten Teilbereich sortieren

Quicksort – Analyse

- Bester und durchschnittlicher Fall haben Laufzeit *O(n * log(n))*
- Schlechtester Fall hat Laufzeit $O(n^2)$
  - Durch geschickte Wahl des Pivotelements kann versucht werden, den schlechtesten Fall zu vermeiden
    - Median of Three: Pivotelement wird aus dem Median von drei Elementen bestimmt (linkes Element, mittleres Element, rechtes Element)
- Es wird kein Hilfsarray benötigt (alle Vertauschungen finden in-place statt)
- Nicht stabiles Sortierverfahren

Vergleich Sortieralgorithmen (1)

- Arrays der Länge *n* mit Zufallszahlen im Bereich *0, …,n*
- Laufzeiten für einen Algorithmus
  - Mittelwert von zehn Messungen pro Eingabegröße
  - Extrapolierte Werte (violett), abgerundete Werte (grün)
- System: Intel Core i7 9750H, Windows 10, Java 14

Vergleich Sortieralgorithmen (2)

Eingabegröße n	Bubblesort (sec)	Insertionsort (sec)	Selectionsort (sec)	Mergesort (sec)	Quicksort (sec)
10.000	0,10	0,01	0,03	0,00	0,00
20.000	0,43	0,05	0,09	0,00	0,00
40.000	1,80	0,19	0,36	0,00	0,00
80.000	7,28	0,79	1,42	0,00	0,00
160.000	29,17	3,14	5,66	0,01	0,01
320.000	117	13	23	0,03	0,02
640.000	468	52	92	0,06	0,05
1.280.000	1872	208	368	0,13	0,10
2.560.000	7488	832	1472	0,26	0,20
5.120.000	29952	3328	5888	0,55	0,43
10.240.000	119808	13312	23552	1,13	0,89

#6 Sortieren in linearer Zeit

Annahmen

Countingsort



Annahmen

- *O(n * log(n))* kann unterboten werden, wenn zusätzliche Annahmen getroffen werden
- Annahmen:
  - Wir gehen von *n* Zahlen im Bereich *0* bis *max* aus
  - `max << n`
  - Hilfsarray `counts` der Länge `max + 1`

Countingsort – Idee

- Zähle in einem Hilfsarray mit, wie oft eine Zahl vorkommt
- Schreibe jede Zahl so oft in das ursprüngliche Array
- Beispiel:

3	2	1	2	0	4	3	2	1	2	2	0	1	3	4	4

Zahl/Index	**0**	**1**	**2**	**3**	**4**
Anzahl	2	3	5	3	3

0	0	1	1	1	2	2	2	2	2	3	3	3	4	4	4

Countingsort – Implementierung

```
private void countingSort(int[] data, int max) {
    int[] counts = new int[max + 1];
    for (int i : data) {
        counts[i]++;
    }
    int i = 0;
    for(int j = 0; j <= max; j++) {
        for(int k = 0; k < counts[j]; k++) {
            data[i++] = j;
        }
    }
}
```

O(n)

O(n)

Countingsort – Analyse

- Eine Schleife für das Befüllen des `counts`-Arrays
  - *O(n)*
- Zwei Schleifen für das Zurückschreiben
  - Es können aber nur n Zahlen geschrieben werden
  - *O(n)*
- Beide Teile werden hintereinander ausgeführt
  - *O(n)*
- Zusätzlicher Speicher notwendig
  - Vernachlässigbar, da `max << n`